

COMSATS University Islamabad
CUI Abbottabad, Dhamtour Campus

AI-Based Network Pathfinding & Attack Simulation System

Course: Artificial Intelligence (CSC 262)

Instructor: Zeenat Zulfiqar

Group Members:

Member 1: [Haseeba Ajmeer]

Member 2: [Iman Nasir]

April 30, 2026

Contents

1	Introduction	2
2	Problem Formulation	2
2.1	Network Graph Model	2
2.2	Network Nodes	2
2.3	Network Edges and Costs	3
2.4	Simulation Parameters	3
3	Algorithm Implementations	3
3.1	BFS — Breadth First Search	3
3.2	DFS — Depth First Search	3
3.3	UCS — Uniform Cost Search	3
3.4	A* Search Algorithm	3
3.5	Heuristic Formulation	4
3.6	Hill Climbing — Local Search	4
3.7	Minimax Algorithm	4
3.8	Alpha-Beta Pruning	4
4	Results and Comparative Analysis	4
4.1	Comparison Table	4
4.2	Analysis	5
5	Conclusion	5

1 Introduction

In modern cybersecurity, attackers move through multiple interconnected systems to reach high-value targets. Understanding these attack paths is critical for building secure networks. This project bridges Artificial Intelligence and Cybersecurity by modeling a computer network as a mathematical graph and applying search algorithms to simulate an attacker's path toward a target system.

The objective of this project is to implement and compare seven AI search algorithms — BFS, DFS, UCS, A*, Hill Climbing, Minimax, and Alpha-Beta Pruning — in the context of network attack path simulation.

2 Problem Formulation

2.1 Network Graph Model

The network is modeled as a **weighted directed graph** , where:

- **Nodes** represent network entities
- **Edges** represent connections between entities
- **Edge Weights** represent vulnerability/risk level

2.2 Network Nodes

Node ID	Type	Description
N0_Attacker	Entry Point	Attacker Machine
N1_WebServer	Server	Public Web Server
N2_Firewall	Security	Firewall
N3_DMZ	Server	DMZ Server
N4_WorkstationA	Workstation	Internal Workstation A
N5_WorkstationB	Workstation	Internal Workstation B
N6_MailServer	Server	Mail Server
N7_Database	Database	Database Server
N8_AdminServer	Server	Admin Server
N9_CriticalAsset	Target	Critical Asset HQ

2.3 Network Edges and Costs

From	To	Cost
N0_Attacker	N1_WebServer	2
N0_Attacker	N3_DMZ	4
N1_WebServer	N2_Firewall	5
N1_WebServer	N3_DMZ	3
N2_Firewall	N4_WorkstationA	4
N3_DMZ	N5_WorkstationB	2
N3_DMZ	N6_MailServer	6
N4_WorkstationA	N7_Database	6
N5_WorkstationB	N6_MailServer	3
N6_MailServer	N8_AdminServer	4
N7_Database	N9_CriticalAsset	5
N8_AdminServer	N9_CriticalAsset	2

2.4 Simulation Parameters

- **Start Node:** N0_Attacker (Attacker Entry Point)
- **Goal Node:** N9_CriticalAsset (Target System)

3 Algorithm Implementations

3.1 BFS — Breadth First Search

BFS explores all nodes level by level using a **queue (FIFO)**. It guarantees the shortest path in terms of number of steps but does not consider edge weights.

3.2 DFS — Depth First Search

DFS explores as deep as possible using a **stack (LIFO)**. It is memory efficient but does not guarantee an optimal path.

3.3 UCS — Uniform Cost Search

UCS expands the node with the **lowest cumulative cost** using a priority queue. It guarantees the optimal path considering edge weights.

3.4 A* Search Algorithm

A* combines actual cost $g(n)$ and heuristic estimate $h(n)$:

$$f(n) = g(n) + h(n) \quad (1)$$

It guarantees the optimal path and is more efficient than UCS.

3.5 Heuristic Formulation

The heuristic $h(n)$ represents the **estimated risk distance** from each node to the goal. Nodes closer to the critical asset have lower heuristic values:

Node	$h(n)$
N0_Attacker	9
N1_WebServer	7
N2_Firewall	6
N3_DMZ	6
N4_WorkstationA	4
N5_WorkstationB	5
N6_MailServer	3
N7_Database	2
N8_AdminServer	2
N9_CriticalAsset	0

3.6 Hill Climbing — Local Search

Hill Climbing greedily selects the neighbor with the best heuristic value. It can get **trapped in a local maximum** and may not reach the goal.

3.7 Minimax Algorithm

Minimax models a two-player game:

- **Attacker (Maximizer):** Tries to reach goal
- **Defender (Minimizer):** Tries to block paths

3.8 Alpha-Beta Pruning

Alpha-Beta Pruning optimizes Minimax by **cutting branches** that cannot affect the final decision, reducing the number of nodes evaluated.

4 Results and Comparative Analysis

4.1 Comparison Table

Algorithm	Path	Cost	Nodes	Time(ms)
BFS	N0→N1→N3→N5→N6→N8→N9	16	8	0.45
DFS	N0→N3→N5→N6→N8→N9	15	6	0.23
UCS	N0→N1→N3→N5→N6→N8→N9	14	9	0.52
A*	N0→N1→N3→N5→N6→N8→N9	14	7	0.41
Hill Climbing	N0→N3→N5→N6→N8→N9	15	6	0.18

Note: Apne actual Colab output se yeh values replace kar lena!

4.2 Analysis

- **BFS** finds shortest hop path but ignores weights
- **DFS** is fastest but not optimal
- **UCS** guarantees optimal cost path
- **A*** is most efficient — optimal path with fewest nodes expanded
- **Hill Climbing** can get stuck at local maxima
- **Alpha-Beta** reduces Minimax calls significantly

5 Conclusion

This project successfully implemented seven AI search algorithms for network attack path simulation. A* proved to be the most efficient algorithm — finding the optimal path while expanding the fewest nodes. Hill Climbing demonstrated the local maxima problem. Alpha-Beta Pruning significantly reduced computation compared to standard Minimax.

Future enhancements could include dynamic network reconfiguration, real-time attack simulation, and integration with actual network security tools.